Grundlagen der IT – Lösungen und Erwartungshorizont

Zu: *aufgaben_git.pdf* (Probeklausuren A und B, Aufgabenpool) · DHBW Heilbronn, WI, 1. Semester · Klausur Mo, 07.12.2026

Hinweise zur Korrektur

- MC: je Option ½ P für richtiges (Nicht-)Ankreuzen, ½ P Abzug für falsches; Minimum 0; kein Kreuz = 0 P. In den Lösungen ist bei jeder Option begründet, warum sie richtig oder falsch ist.
- Rechenaufgaben: Ergebnis ohne Rechenweg nur Teilpunkte. Folgefehler werden berücksichtigt.
- Skizzen: Die SVG-Grafiken sind Musterskizzen; andere sinnvolle Anordnungen sind gleichwertig, solange die geforderten Elemente beschriftet sind.
- Alle Rechnungen, Bäume, Sortierschritte und SQL-Ergebnisse wurden mit Python bzw. SQLite automatisch nachgeprüft.

Bekannte Unsicherheiten

- Binäre Suche: Die Folie zeigt für die 19 eine Variante mit drei Vergleichen (obere Mitte). Hier gilt durchgängig Mitte = ⌊(l + r)/2⌋; in der Klausur die verwendete Regel dazuschreiben.
- Löschen im Binärbaum mit zwei Nachfolgern: Die Folie erlaubt „größter links“ oder „kleinster rechts“; beide Varianten sind angegeben, wo die Aufgabe es offenlässt.
- Gleitkomma (IEEE 754), Shift-Operator und binäre Kommazahlen stehen auf den Folien als Exkurs; die Klausurvorbereitung nennt „Abbildung nicht ganzzahliger Werte“ – daher nur im Pool.
- Bubble Sort: Laut Folie „so viele Durchläufe wie Elemente“; hier wird abgebrochen, sobald ein Durchlauf ohne Vertauschung bleibt (Ergebnis identisch).

Erwartungshorizont Probeklausur A (50 P)

Aufgabe 1: Information und Digitalisierung — 3 Punkte

a) Daten sind Rohdaten bzw. unverarbeitete Fakten (Zahlen, Texte, Symbole) ohne Bedeutung. **Information** sind Daten, die durch einen **Kontext** eine Bedeutung erhalten; sie unterstützen Entscheidungen, verdichten sich zu Wissen, und Wissen führt zu Handlung.

Beispiel: `0000 0111` = 7 (Datum). Kontext „Temperatur in °C in Deutschland“ → Information „Es ist 7 Grad“ → Wissen „Es ist kalt“ → Handlung „Jacke anziehen“. Im Kontext „°F in den USA“ bedeutet dieselbe 7 etwa −13 °C → Wintersachen.

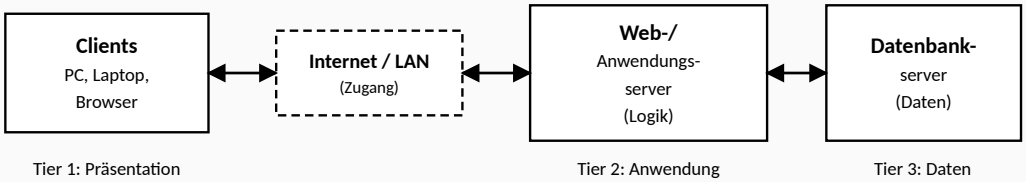
b) Automatisierung · Informatisierung · Transformation.

Punktvergabe: a) Daten ½, Information/Kontext ½, Beispiel mit Kontext ½. b) je Stufe ½ (eine Erläuterung ist nicht verlangt).

Aufgabe 2: IT-Infrastruktur und Cloud-Computing — 4 Punkte

a) vier aus: Hardwareplattformen · Betriebssystem-Plattformen · Datenverwaltung und -speicherung · Netzwerke und Telekommunikation · Anwendungssysteme · Internet (als Plattform) · Beratungsdienste und Systemintegratoren.

b) Desktop- oder Laptop-Rechner (**Clients**) sind mit leistungsstarken **Servern** vernetzt. Der Client ist die Eingabestelle des Benutzers; der Server verarbeitet und speichert gemeinsam genutzte Daten, liefert Webseiten oder verwaltet Netzwerkaktivitäten. Bei einer **N-Tier-Architektur** (mehrschichtig) wird die Arbeit auf mehrere Server-Ebenen verteilt, z. B. Client → Web-/Anwendungsserver → Datenbankserver (SAP R/3 mit Internet Transaction Server und separatem Datenbankserver). So verteilt man die Rechenlast auf viele kleinere, günstigere Maschinen.



Punktvergabe: a) je Komponente ½ (max. 2). b) Client-Server-Prinzip mit Rollen 1 P, N-Tier (mehrere Server-Ebenen, Beispiel) 1 P.

Aufgabe 3: Mooresches Gesetz — 2 Punkte

- ☒ Gordon Moore sagte 1965 eine jährliche Verdopplung der Anzahl der Komponenten (Transistoren) auf einem Chip voraus.
  richtig: So steht es auf der Folie; später reduzierte Moore die Rate auf eine Verdopplung alle zwei Jahre.
- ☒ Eine gängige Interpretation lautet: Die Kosten für rechnergestützte Informationsverarbeitung halbieren sich etwa alle 18 Monate.
  richtig: Eine der drei Interpretationen auf der Folie (keine davon stammt von Moore selbst).
- ☐ Das Mooresche Gesetz besagt, dass der Wert eines Netzwerks überproportional mit der Zahl seiner Teilnehmer wächst.
  falsch: Das ist das Gesetz von Metcalfe (n·(n−1) mögliche Verbindungen).
- ☐ Die Anzahl der Transistoren auf einem Chip verdreifacht sich etwa alle 18 Monate.
  falsch: Es geht um eine Verdopplung, nicht um eine Verdreifachung (Falle aus dem Folienbeispiel).

Punktvergabe: 4 Optionen × ½ P; negative Summe → 0.

Aufgabe 4: Zahlendarstellung und Rechnen — 3 Punkte

a) Restwertmethode (Reste von unten nach oben lesen):

```
173 : 2 = 86 Rest 1
86 : 2 = 43 Rest 0
43 : 2 = 21 Rest 1
21 : 2 = 10 Rest 1
10 : 2 = 5 Rest 0
5 : 2 = 2 Rest 1
2 : 2 = 1 Rest 0
1 : 2 = 0 Rest 1
```

173_{10} = **$1010\ 1101_2$**. Oktal: Dreiergruppen von rechts `010 101 101` → **255_8**. Hex: Vierergruppen `1010 1101` → **AD_{16}** (Probe: 2·64 + 5·8 + 5 = 173; 10·16 + 13 = 173).

b) 23 = `00010111`. 57 = `00111001` → invertieren `11000110` → +1 → −57 = `11000111`.

```
  00010111   (+23)
+ 11000111   (−57)
= 11011110   (kein Übertrag aus Bit 7)
```

Ergebnis `11011110`: höchstes Bit = 1 → negativ. Probe: −128 + 64 + 16 + 8 + 4 + 2 = **-34** ✓ (bzw. invertieren + 1 → 0010 0010 = 34). Kein Überlauf, da −34 im Bereich −128 … +127 liegt.

Punktvergabe: a) je System ½ (nur mit Rechenweg volle Punkte). b) −57 korrekt ½, Addition ½, Rückrechnung/Interpretation ½.

Aufgabe 5: Wahrheitstabelle, Boolesche Ausdrücke und Logikgatter — 4 Punkte

a)

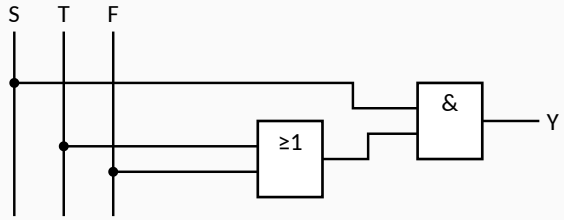
S	T	F	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

b) $Y = S \wedge (T \vee F)$

gleichwertig als disjunktive Normalform (Zeilen mit Y = 1):

$Y = (S \wedge \neg T \wedge F) \vee (S \wedge T \wedge \neg F) \vee (S \wedge T \wedge F)$

c) (IEC-60617-Symbole; ANSI-Symbole ebenfalls zulässig)



Punktvergabe: a) 8 korrekte Zeilen 2 P, je fehlerhafte Zeile −½. b) 1 P (vereinfacht oder DNF). c) 1 P, Folgefehler aus b) werden berücksichtigt.

Aufgabe 6: Programmierung — 2 Punkte

- [x] Ein Compiler übersetzt ein Quellprogramm vorab in Maschinenbefehle für eine bestimmte Hardware.
  richtig: Definition „Compilieren“ (im Gegensatz zum Interpreter, der zur Laufzeit übersetzt).
- [ ] Semantikfehler werden immer vom Compiler gemeldet.
  falsch: Der Compiler meldet Syntaxfehler; ein Semantikfehler (falsche Bedeutung) läuft oft ohne Meldung durch.
- [x] Ein Algorithmus muss in endlicher Zeit zum Ziel kommen, ein Programm darf dagegen auch endlos laufen (z. B. ein Serverprozess).
  richtig: So auf den Folien zu Algorithmus / Programm / Prozess.
- [ ] Assembler ist eine höhere, prozessorunabhängige Programmiersprache.
  falsch: Assembler gibt Maschinenbefehlen nur symbolische Namen und ist prozessorabhängig.

Punktvergabe: 4 Optionen × ½ P.

Aufgabe 7: Datenstrukturen — 8 Punkte

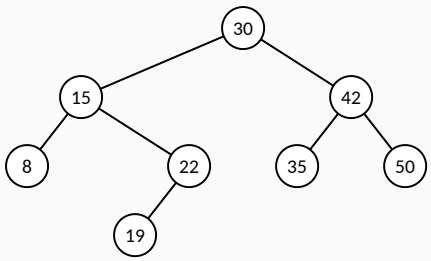
a)

Operation	Rückgabe	Stapel (unten → oben)
push(3)	–	[3]
push(7)	–	[3, 7]
pop()	7	[3]
push(5)	–	[3, 5]
push(9)	–	[3, 5, 9]
top()	9	[3, 5, 9]
pop()	9	[3, 5]
push(1)	–	[3, 5, 1]

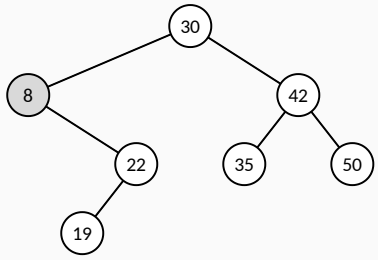
Rückgaben: pop() → 7, top() → 9, pop() → 9. Endzustand (unten → oben): 3, 5, **1 (oben)**.

b) FIFO (First In, First Out); `put()` hängt am Ende an, `get()` entnimmt am Anfang (englisch auch enqueue/dequeue). Fehlerfall: get() auf leere Queue.

c)



d) 15 hat zwei Nachfolger → größter Knoten links ist 8 (Blatt), er rückt an die Stelle von 15:



(Mit der Alternative „kleinster Knoten rechts“ entstünde 30(19(8, 22), 42(35, 50)); hier war aber „größter links“ verlangt.)

e) Nein. Knoten 22 hat nur einen Nachfolger (19), ist also ein „Halbblatt“. Ein voller Binärbaum hat keine Knoten mit genau einem Nachfolger. (Damit ist er auch nicht vollständig.)

f) Array: direkter, schneller Zugriff über den Index; einfach. Liste: dynamische Länge, Einfügen/Löschen durch Umhängen der Zeiger ohne Umkopieren.

Punktvergabe: a) Rückgabewerte 1 P, Endzustand 1 P. b) FIFO ½, put/get ½. c) korrekter Baum 2 P (je falsch platzierter Knoten −½). d) 1 P (Folgefehler berücksichtigt). e) Nein + Begründung 1 P. f) je ½.

Aufgabe 8: Suchen und Sortieren — 6 Punkte

a)

	Array	Vertauschungen
nach Durchlauf 1	[10, 14, 29, 13, 37]	3
nach Durchlauf 2	[10, 14, 13, 29, 37]	1
nach Durchlauf 3	[10, 13, 14, 29, 37]	1
nach Durchlauf 4	[10, 13, 14, 29, 37]	0

Nach dem 1. Durchlauf steht das größte Element (37) am Ende; im 4. Durchlauf gibt es keine Vertauschung mehr → sortiert.

b) Bereich 0…8, Mitte 4: **23** → Bereich 5…8, Mitte 6: **40** → Bereich 7…8, Mitte 7: **52**. Verglichen werden also 23 (52 > 23 → rechts), 40 (52 > 40 → rechts), 52 (gefunden) – 3 Vergleiche.

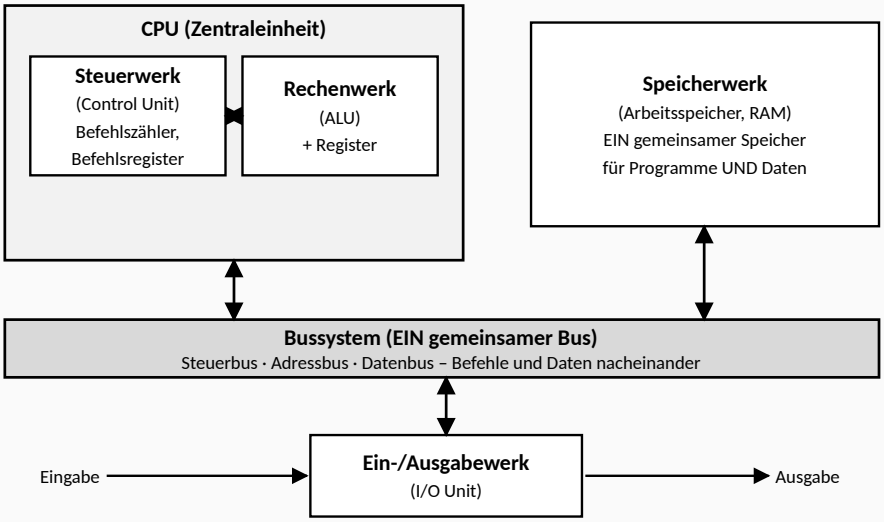
c)

- [x] Die lineare Suche funktioniert auch auf unsortierten Daten.
  richtig: Alle Elemente werden nacheinander geprüft – Sortierung ist nicht nötig, Aufwand O(n).
- [ ] Merge Sort hat einen Aufwand von $O(n^2)$.
  falsch: Merge Sort: O(n log n). $O(n^2)$ ist Bubble Sort.
- [x] Merge Sort folgt dem Prinzip „Teile und Herrsche“ und wird typischerweise rekursiv implementiert.
  richtig: Halbieren, Hälften (rekursiv) sortieren, dann zusammenführen.
- [ ] Die binäre Suche hat einen Aufwand von O(n).
  falsch: Binäre Suche: O(log n), begrenzt durch die Anzahl der Halbierungen.

Punktvergabe: a) Durchläufe 1–3 korrekt 1,5 P, Abbruch/Endergebnis ½. b) je verglichenes Element ½, Ergebnis/Richtung ½. c) 4 × ½.

Aufgabe 9: Rechnerarchitektur — 8 Punkte

a) Musterskizze:



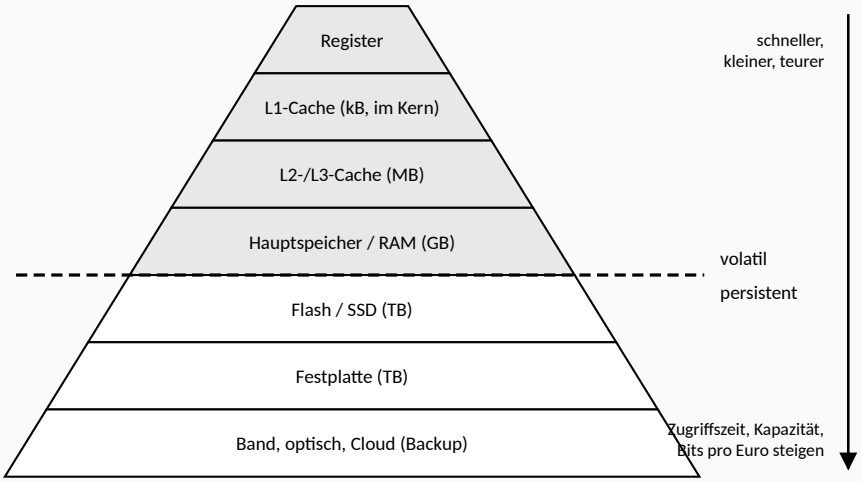
b) 1. FETCH (Befehl aus dem Speicher ins Befehlsregister laden) → 2. DECODE (Steuerwerk löst den Befehl in Schaltinstruktionen auf) → 3. FETCH OPERANDS (Operanden aus dem Speicher holen) → 4. EXECUTE (Rechenwerk führt aus, ggf. Ergebnis speichern) → 5. UPDATE INSTRUCTION POINTER (Befehlszähler erhöhen bzw. bei Sprung neu setzen); dann beginnt der Zyklus von vorn.

c) Beim Von-Neumann-Rechner liegen Programme und Daten im selben Speicher und werden über denselben Bus transportiert. Befehl und Daten müssen deshalb **nacheinander** über den gemeinsamen Bus geholt werden; eine Anweisung braucht mindestens zwei Taktzyklen, und die schnelle CPU wartet auf den Speicher. Die Harvard-Architektur verwendet **getrennte Speicher** (separate Adressen) und **getrennte Busse** für Befehle und Daten. So können Befehl und Daten in einem Schritt gleichzeitig geholt werden (Einzelzyklus), die Ausführung ist schneller. Nachteil: in der Regel **teurer** bzw. aufwendiger zu implementieren.

Punktvergabe: a) je Komponente ½ (Steuerwerk, Rechenwerk, Speicherwerk, E/A-Werk, Bussystem) = 2,5; ½ für „ein gemeinsamer Speicher für Programme und Daten / ein Bus“. b) je Phase in richtiger Position ½. c) Flaschenhals 1 P, Harvard-Lösung 1 P, Nachteil ½.

Aufgabe 10: Speicher — 4 Punkte

a) Register → L2-Cache → Hauptspeicher (RAM) ‖ *volatil / persistent* ‖ Flash/SSD → Festplatte → Bandlaufwerk.



b)

RAID-Level	nutzbare Kapazität	Ausfalltoleranz
RAID 0	12 TB	0 Platte(n)
RAID 5	9 TB	1 Platte(n)
RAID 6	6 TB	2 Platte(n)

RAID 0: alle 4 × 3 TB, keine Redundanz. RAID 5: n − 1 Platten nutzbar (Parität verteilt). RAID 6: n − 2 Platten nutzbar.

c) Größere Datenblöcke werden in einzelne Abschnitte (Strips) zerlegt und reihum auf mehrere parallele Platten verteilt; beim Lesen werden alle Platten parallel angesprochen → höhere Datenrate.

Punktvergabe: a) Reihenfolge 1,5 P (je Fehler −½), Grenze ½. b) je Zeile ½. c) ½.

Aufgabe 11: Datenbanken — 6 Punkte

a) zwei aus: Redundanzkontrolle (Daten nur einmal gespeichert, Sichten/Views) · Zugriffsbeschränkungen · persistente Speicherung · Mehrbenutzerzugriff/-schnittstellen · Überwachung von Beziehungen und Integritätsbedingungen · Recovery nach Fehlern · Datenunabhängigkeit (Data Dictionary, interne Speicherung verborgen).

b1)

Name	Gehalt
Becker	4200
Neumann	4900

b2)

AbtNr	Anzahl	MAX(Gehalt)
10	3	4900
20	3	5100
30	1	4600

c1)

```
SELECT MITARBEITER.Name, ABTEILUNG.Bezeichnung
FROM   MITARBEITER, ABTEILUNG
WHERE  MITARBEITER.AbtNr = ABTEILUNG.AbtNr
  AND  ABTEILUNG.Standort = 'Heilbronn';
```

Ergebnis zur Kontrolle:

Name	Bezeichnung
Becker	Vertrieb
Wagner	Controlling
Kaya	Vertrieb
Neumann	Vertrieb

c2)

```
SELECT AbtNr, AVG(Gehalt)
FROM   MITARBEITER
GROUP BY AbtNr;
```

Ergebnis zur Kontrolle (Abt. 10: 12700 / 3 ≈ 4233,33):

AbtNr	AVG(Gehalt)
10	4233,33
20	4600
30	4600

Punktvergabe: a) je Vorteil ½. b1) 1 P (beide Zeilen). b2) 1 P (je falsche Zeile −½). c1) SELECT/FROM ½, Join-Bedingung ½, Standort-Filter ½. c2) AVG ½, GROUP BY ½, sinnvolle Spaltenliste ½. Gleichwertige Lösungen (JOIN … ON, Aliasse) volle Punkte.

Erwartungshorizont Probeklausur B (49 P)

Aufgabe 1: Information und Digitalisierung — 3 Punkte

a) betriebswirtschaftliche Sichtweise (Daten und algorithmische Systeme für neue/bessere Prozesse, Produkte, Geschäftsmodelle) · gesellschaftliche Sichtweise (Übertragung von Lebens- und Arbeitswelten auf eine digitale Ebene, „Disruption“) · technische Sichtweise (Überführung analoger in zeit- und wertediskrete, binäre Signale zum Übertragen, Speichern, Verarbeiten).

b) (1) Phase 4: individuelle 1:1-Integration überbetrieblicher Prozesse · (2) Phase 1: Digitalisierung einzelner Funktionen · (3) Phase 3: integrierte Prozesse über Grenzen der Aufbauorganisation hinweg.

Punktvergabe: a) je Sichtweise ½. b) je korrekter Zuordnung ½.

Aufgabe 2: IT-Infrastruktur und Cloud-Computing — 4 Punkte

a)

- [x] Beim Cloud Computing werden vormals im Unternehmen betriebene Lösungen von externen Anbietern über das Internet bezogen.
  richtig: Definition laut Folie (IaaS, PaaS, SaaS).
- [x] Salesforce (CRM) gilt als Pionier für „Software-as-a-Service“.
  richtig: Beispiel auf der Folie zur 5. Epoche.
- [ ] Programme und Daten verbleiben beim Cloud Computing grundsätzlich auf lokalen Servern des Unternehmens.
  falsch: Gerade nicht: sie werden über das Internet bezogen, meist im Abo, Zugang i. d. R. per Browser.
- [ ] Mit Beginn der Cloud-Epoche wurde das Client-Server-Computing vollständig abgelöst und wird heute nicht mehr eingesetzt.
  falsch: Die Epochen überlagern sich (Client-Server: 1983 – heute).

b) 1. Groß- und Minirechner (1959 – heute) → 2. Personal Computing (1981 – heute) → 3. Client-Server-Computing (1983 – heute) → 4. betriebliche und internetbasierte Informationssysteme (1992 – heute) → 5. Cloud Computing (2000 – heute). Jahreszahlen sind nicht verlangt.

Punktvergabe: a) 4 × ½. b) alle fünf korrekt und chronologisch 2 P; je fehlende oder falsch eingeordnete Epoche −½.

Aufgabe 3: Mooresches Gesetz — 2 Punkte

- [x] Moore reduzierte seine ursprüngliche Prognose später auf eine Verdopplung der Transistoren etwa alle zwei Jahre.
  richtig: Folie: zunächst jährlich (1965), später alle zwei Jahre.
- [ ] Alle gängigen Interpretationen (z. B. „Leistung verdoppelt sich alle 18 Monate“) stammen von Gordon Moore selbst.
  falsch: Folie: mindestens drei Interpretationen, von denen keine von Moore selbst stammt.
- [x] Die Fläche ist der wichtigste Kostenfaktor bei Mikrochips; mehr Transistoren pro Fläche senken daher die Kosten der Informationsverarbeitung.
  richtig: Folie „Konsequenzen & Interpretationen“.
- [ ] Das Mooresche Gesetz beschreibt ein lineares Wachstum der Transistorzahl um eine feste Anzahl pro Jahr.
  falsch: Es beschreibt exponentielles Wachstum (Verdopplung).

Punktvergabe: 4 × ½ P.

Aufgabe 4: Zahlendarstellung und Rechnen — 3 Punkte

a) Hex (Vierergruppen): `1011 0110 1101` → B 6 D → **B6D$_{16}$**. Oktal (Dreiergruppen): `101 101 101 101` → **5555$_{8}$**. Dezimal: 11·256 + 6·16 + 13 = 2816 + 96 + 13 = **2925** (Probe oktal: 5·512 + 5·64 + 5·8 + 5 = 2925).

b) 45 = `00101101` → −45 = `11010011`; 90 = `01011010` → −90 = `10100110`.

```
  11010011   (−45)
+ 10100110   (−90)
= 101111001   → 8 Bit: 01111001
```

Das 8-Bit-Ergebnis `01111001` wäre **+121** – zwei negative Zahlen ergeben eine positive: **Überlauf**. Das richtige Ergebnis −135 liegt außerhalb des Wertebereichs −128 … +127 und ist mit 8 Bit nicht darstellbar.

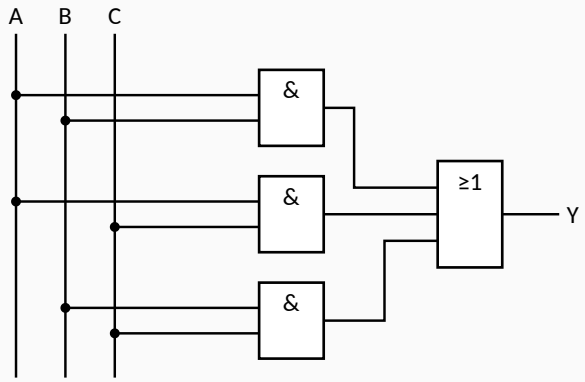
Punktvergabe: a) je System ½. b) beide Operanden korrekt ½, Addition ½, Erkennen und Begründen des Überlaufs ½.

Aufgabe 5: Wahrheitstabelle, Boolesche Ausdrücke und Logikgatter — 4 Punkte

a) Y = (¬A ∧ B ∧ C) ∨ (A ∧ ¬B ∧ C) ∨ (A ∧ B ∧ ¬C) ∨ (A ∧ B ∧ C)

Vereinfacht (nicht verlangt): Y = (A ∧ B) ∨ (A ∧ C) ∨ (B ∧ C) – Y ist 1, wenn mindestens zwei Eingänge 1 sind („Mehrheitsentscheider“).

b) Schaltung zum vereinfachten Ausdruck (die DNF mit vier UND-Gattern mit je drei Eingängen und NOT-Gattern ist ebenso richtig):



c) XNOR („gleich“, NOT XOR):

A	B	A XNOR B
0	0	1
0	1	0
1	0	0
1	1	1

Punktvergabe: a) 4 Terme korrekt 2 P (je falscher/fehlender Term −½). b) 1 P (Folgefehler berücksichtigt). c) XNOR ½, Tabelle ½.

Aufgabe 6: Programmierung — 2 Punkte

a) Syntax = korrekte Schreibweise; ein Syntaxfehler (z. B. fehlende Klammer `int x = (3 + 4;`) wird vom Compiler gemeldet. **Semantik** = korrekte Bedeutung; ein Semantikfehler wird nicht gemeldet, das Ergebnis ist trotzdem falsch (z. B. `double res = 10 / 3;` liefert in Java 3.0 statt 3,33, oder Variablennamen passen nicht zu den Werten).

b) Eine Funktion ruft sich selbst (ggf. mehrfach) mit geänderten, kleineren Eingabewerten auf (Implementierung von „Teile und Herrsche“, z. B. N!, Merge Sort). Sie braucht eine **Abbruchbedingung** (z. B. 0! = 1! = 1), sonst ruft sie sich endlos auf.

Punktvergabe: a) Unterschied ½, Beispiele ½. b) Selbstaufruf ½, Abbruchbedingung ½.

Aufgabe 7: Datenstrukturen — 7 Punkte

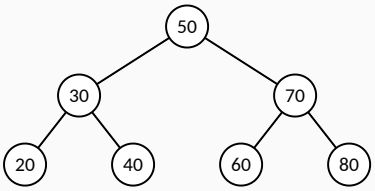
a)

Operation	Rückgabe	Queue (Anfang → Ende)
put(4)	–	[4]
put(9)	–	[4, 9]
get()	4	[9]
put(2)	–	[9, 2]
put(7)	–	[9, 2, 7]
get()	9	[2, 7]
put(5)	–	[2, 7, 5]

Rückgaben: get() → 4, get() → 9. Endzustand: **Anfang → 2, 7, 5** (FIFO).

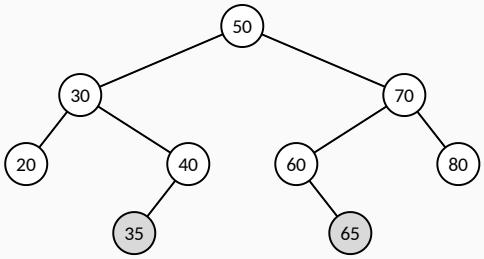
b) Liste ab dem Startelement durchlaufen, bis der Vorgänger gefunden ist (11: letztes Element < 16). Neues Element 16 anlegen; zuerst 16.next := 25 (Nachfolger von 11), dann 11.next := 16. Die Sortierung bleibt erhalten, nichts wird umkopiert.

c)



Höhe 2 (höchste Ebene; Ebene = Anzahl Kanten zur Wurzel). Der Baum ist **voll** (keine Halbblätter) und **vollständig** (alle Blätter auf der letzten Ebene).

d)



Nicht mehr voll: 40 und 60 haben jetzt nur einen Nachfolger.

e) Jeder Knoten hat höchstens einen Nachfolger, der Baum entartet zu einer Liste (Suche O(n) statt O(log n)). Das passiert z. B., wenn die Werte bereits sortiert eingefügt werden (20, 30, 40, …).

Punktvergabe: a) Rückgaben 1 P, Endzustand 1 P. b) Vorgänger suchen ½, Zeiger in richtiger Reihenfolge umhängen ½. c) Baum 1 P, Höhe ½, voll + vollständig ½. d) Baum ½, „nein“ + Begründung ½. e) 1 P.

Aufgabe 8: Suchen und Sortieren — 6 Punkte

a)

```
Teilen 0: [38 27 43 3 9 82 10 5]
Teilen 1: [38 27 43 3]  [9 82 10 5]
Teilen 2: [38 27]  [43 3]  [9 82]  [10 5]
Teilen 3: [38]  [27]  [43]  [3]  [9]  [82]  [10]  [5]
Mischen 1: [27 38]  [3 43]  [9 82]  [5 10]
Mischen 2: [3 27 38 43]  [5 9 10 82]
Mischen 3: [3 5 9 10 27 38 43 82]
```

Teile und Herrsche: halbieren bis zu Einzelelementen, dann jeweils zwei sortierte Teillisten durch Vergleich der vordersten Elemente zusammenführen.

b) Linear: bis zu **1000** Vergleiche (jedes Element, O(n)). Binär: Suchbereich wird je Schritt halbiert; $2^{10} = 1024 \geq 1000 \rightarrow$ etwa **10** Vergleiche (O(log n)).

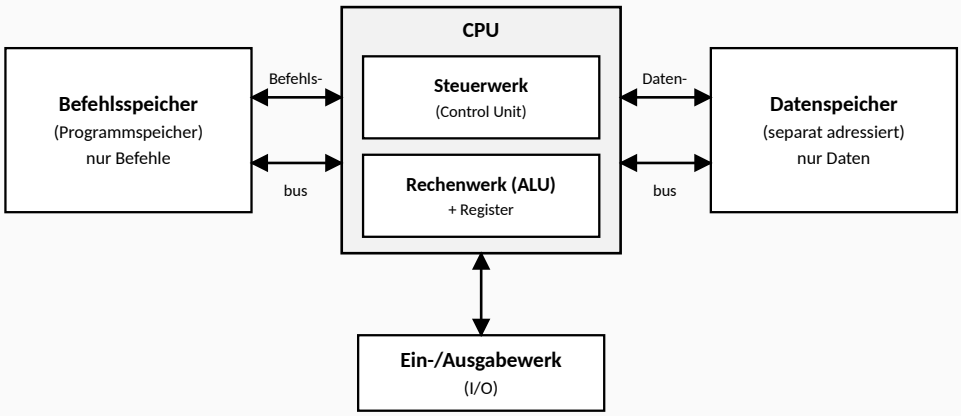
c)

- [x] Bubble Sort benötigt im ungünstigsten Fall so viele Durchläufe wie Elemente vorhanden sind; der Aufwand ist $O(n^2)$.
  richtig: Folie Bubble Sort.
- [ ] Nach dem ersten Durchlauf von Bubble Sort (aufsteigend) steht das kleinste Element sicher am Anfang.
  falsch: Sicher ist nur: das größte Element steht am Ende.
- [ ] Die binäre Suche kann auch auf eine unsortierte Liste angewendet werden.
  falsch: Binäre Suche setzt sortierte Daten voraus.
- [x] Bei Merge Sort werden die beiden bereits sortierten Hälften abschließend zusammengeführt („Merge“).
  richtig: Folie Merge Sort.

Punktvergabe: a) Teilen 1 P, Mischen 2 P (je Ebene ⅔). b) beide Zahlen mit Begründung je ½. c) 4 × ½.

Aufgabe 9: Rechnerarchitektur — 8 Punkte

a) Musterskizze:



Zwei getrennte Speicher + zwei getrennte Busse → Befehl und Daten gleichzeitig (ein Schritt)

b)

	Von-Neumann	**Harvard**
Speicher	ein gemeinsamer Speicher für Programme und Daten (binär kodiert)	getrennte Speicher (separate Adressen) für Befehle und Daten
Busse	ein gemeinsamer Bus; Befehle und Daten nacheinander	getrennte Busse für Befehle und Daten; gleichzeitig in einem Schritt
Vorteil	einfachere Implementierung, geringere Kosten; Programme zur Laufzeit ladbar	schneller (Einzelzyklus), kein Von-Neumann-Flaschenhals
Nachteil	Von-Neumann-Flaschenhals (≥ 2 Takte je Anweisung); Sicherheit: als Daten getarnter Code / Buffer Overflow	in der Regel teurer, aufwendiger

Einsatz Harvard (Zusatzwissen): Mikrocontroller, Signalverarbeitung, Spezialrechner für maschinelles Lernen, zunehmend PCs/Server.

c) Steuerbus: Lesebefehl · Adressbus: Adresse der Speicherzelle · Datenbus: Daten von der Speicherzelle zur CPU.

Punktvergabe: a) CPU (Steuer-/Rechenwerk) ½, zwei getrennte Speicher 1, zwei getrennte Busse ½, E/A ½. b) je Zelle ½. c) je Bus ½.

Aufgabe 10: Speicher — 4 Punkte

a) 16 GB = $2^4 \cdot 2^{30}$ Byte = 2^{34} Byte → **34** Adressleitungen. 20 Leitungen → 2^{20} Byte = 1.048.576 Byte = **1 MB**.

b) Programme greifen in kurzer Zeit meist nur auf einen kleinen, benachbarten Speicherbereich zu (Lokalität: Schleifen, Arrays). Deshalb werden beim Zugriff auf ein Wort auch die Nachbarn in den schnellen Cache geladen; die folgenden Zugriffe treffen meist den Cache, statt über den langsamen Bus zum RAM zu gehen. Nur bei einem Cache-Miss („Cache-fault“) muss nachgeladen werden.

c) RAID 5: (6 − 1) · 2 TB = **10 TB**, 1 Platte darf ausfallen. RAID 6: (6 − 2) · 2 TB = **8 TB**, 2 Platten dürfen ausfallen.

d) RAID 0 (Striping) verteilt die Daten ohne Redundanz auf alle Platten – fällt eine aus, sind die Daten verloren. Vorteil: hoher Datendurchsatz, volle Kapazität (z. B. für temporäre Daten).

Punktvergabe: a) je Teilergebnis ½. b) Lokalität ½, Nachbarn im Cache / weniger Buszugriffe ½. c) je RAID-Level ½. d) keine Redundanz ½, Durchsatz ½.

Aufgabe 11: Datenbanken — 6 Punkte

a) Primärschlüssel: Attribut, dessen Wert jeden Datensatz eindeutig identifiziert (KUNDE.KNr, BESTELLUNG.BestNr). **Fremdschlüssel**: Attribut, das auf den Primärschlüssel einer anderen Tabelle verweist (BESTELLUNG.KNr → KUNDE.KNr). Kunde – Bestellung ist **1:n** (ein Kunde, viele Bestellungen; jede Bestellung gehört zu genau einem Kunden) – der Fremdschlüssel steht auf der n-Seite.

b)

Name	Anzahl
Müller	3
Öztürk	2

Fischer erscheint **nicht**: Sie hat keine Bestellung, die Join-Bedingung KUNDE.KNr = BESTELLUNG.KNr ist für sie nie erfüllt (Kunden ohne Bestellungen werden nicht ausgegeben).

c1)

```
SELECT BestNr, Datum, Betrag
FROM   BESTELLUNG
WHERE  Betrag > 100;
```

BestNr	Datum	Betrag
501	2026-10-02	120
504	2026-10-12	210
506	2026-10-23	150

c2)

```
SELECT KUNDE.Name, SUM(BESTELLUNG.Betrag) AS Umsatz
FROM   KUNDE, BESTELLUNG
WHERE  KUNDE.KNr = BESTELLUNG.KNr
GROUP BY KUNDE.Name;
```

Name	Umsatz
Müller	269,90
Schmidt	210
Weber	35
Öztürk	195,50

c3)

```
INSERT INTO KUNDE (KNr, Name, Ort)
VALUES (6, 'Klein', 'Heilbronn');
```

Punktvergabe: a) Primärschlüssel ½, Fremdschlüssel ½, 1:n ½. b) Ergebnis 1 P, Begründung Fischer ½. c1) 1 P. c2) Join ½, SUM + GROUP BY ½. c3) 1 P (Spaltenliste optional).

Block 1: Lösungen Information und Digitalisierung

Klausur-Aufgabe 1 (3 P)

P1.1

Daten	Information	Wissen	Handlung
0010 1010 = 42	Ich habe 42 €	Das reicht für die Kinokarte (14 €)	Ich gehe ins Kino
0000 0111 = 7	Es ist 7 °F	≈ -14 °C → sehr kalt (Folie: −13 °C)	Wintersachen anziehen

Kernaussage: Dieselben Daten ergeben je nach Kontext unterschiedliche Information (7 °C in Deutschland → Jacke).

P1.2

- **Automatisierung:** bisher manuelle Abläufe werden „in IT gegossen" → schneller und standardisiert (z. B. automatische Rechnungserstellung).
- **Informatisierung:** die dabei entstehenden digitalen Daten werden ausgewertet und unterstützen Managemententscheidungen (Business Intelligence, Data Warehouse, Business Analytics).
- **Transformation:** völlig neue Geschäftsprozesse, die vorher an Medienbrüchen oder begrenzter Rechenkapazität scheiterten (Business Process Reengineering, z. B. digitale Plattformen).

P1.3

☒ Aus technischer Sicht bedeutet Digitalisierung die Überführung analoger in zeit- und wertediskrete (binäre) Signale.
richtig: Technische Sichtweise.

☒ In der heutigen Phase der Digitalisierung werden überbetriebliche Prozesse durchgängig (m:n) bis zum Endkunden integriert.
richtig: Phase „heute" nach Österle/Fleisch/Alt.

☒ Uber und Airbnb sind Beispiele für Plattform-Geschäftsmodelle ohne eigene Fahrzeuge bzw. Immobilien.
richtig: Ökonomische Aspekte der Digitalisierung.

☐ Laut Carr („IT doesn't matter", 2003) war IT schon immer ein strategischer Wettbewerbsvorteil.
falsch: Carr stellte gerade in Frage, ob IT Wettbewerbsvorteile bringt; prä-Internet galt IT als nicht strategisch.

☒ „Freemium" bedeutet: Basisversion kostenlos (oder werbefinanziert), Premiumversion gegen Bezahlung/Abo.
richtig: Folie, Beispiel Fernlichtassistent als Cloud-Abo.

☐ Die fünf Epochen der IT-Infrastruktur sind identisch mit den Phasen der Digitalisierung.
falsch: Ausdrücklich „nicht zu verwechseln": Epochen = technische Sicht, Phasen = ökonomische Sicht.

P1.4

Vor dem Internet war IT eine untergeordnete Managementaufgabe der „EDV-Abteilung". Sie galt als notwendig, um Routineabläufe effizienter zu machen, aber nicht als strategisch relevant (Carr 2003: „IT doesn't matter"). Mit dem kommerziellen Internet betraf IT unmittelbar die wertschöpfenden Aktivitäten und wurde Teil des Kerngeschäfts: Kommunikations- und Vertriebskanäle wurden digital (CRM, Shopsysteme), Produkte wurden digitalisiert oder neu geschaffen (z. B. Spotify). Heute kommen standardisierte Betriebsmodelle wie Cloud Computing (SaaS, PaaS) hinzu, und strategische Vorteile entstehen aus Data Analytics, KI und Machine Learning.

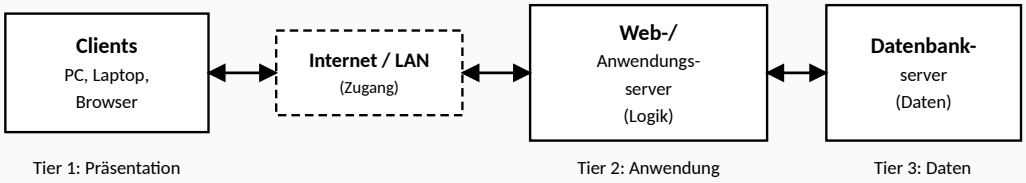
Block 2: Lösungen IT-Infrastruktur und Cloud-Computing

Klausur-Aufgabe 2 (4 P)

P2.1

Element	Hauptkomponente
SAP S/4HANA (ERP-System)	Anwendungssysteme
Windows Server und Linux	Betriebssystem-Plattformen
Oracle-Datenbank, NAS-Speicher	Datenverwaltung und -speicherung
Switches, Router, WLAN, Telefonanlage	Netzwerke und Telekommunikation
Server und Laptops eines Hardwareherstellers	Hardwareplattformen
Webshop und REST-Schnittstellen, die über das Internet erreichbar sind	Internet als Plattform
externes Beratungshaus, das das ERP-System einführt	Beratungsdienste und Systemintegratoren

P2.2



Vorteil: Die Rechenlast wird auf mehrere kleinere, kostengünstige Maschinen verteilt; einzelne Schichten lassen sich getrennt ausbauen.

P2.3

Epoche	seit	typisches Merkmal
1. Groß- und Minirechner	1959	IBM 1401/7090, tausende Terminals an einem Großrechner, alles von einem Anbieter; DEC-Minicomputer (PDP-11, VAX) dezentralisieren
2. Personal Computing	1981	IBM-PC, „Wintel“-Standard, Tabellenkalkulation/Textverarbeitung; Vernetzung erst um 1990
3. Client-Server	1983	Clients als Eingabestelle, Server für gemeinsame Daten; N-Tier, z. B. SAP R/3
4. betriebliche und internetbasierte Informationssysteme	1992	TCP/IP verbindet heterogene Netze, unternehmensübergreifende Kommunikation, REST-APIs, Supply Chain
5. Cloud Computing	2000	Lösungen über das Internet bezogen (IaaS/PaaS/SaaS), Abo, Browser, Hyperscaler

Die Epochen überlagern sich – alle laufen „bis heute“.

P2.4

a) Vormals im Unternehmen betriebene Lösungen werden von externen Anbietern über das Internet bezogen. Programme und Daten liegen nicht mehr auf lokalen Ressourcen, sie werden meist im Abo bezogen und über den Browser genutzt. Anbieter stellen flexible, skalierbare virtuelle Instanzen bereit (Beispiel: New York Times wandelte mit gemieteten AWS-Instanzen ca. 11 Mio. Artikel in 24 h in PDFs um).

b) Amazon musste für Weihnachts- und „Back2School“-Geschäft hohe Rechenleistung vorhalten und suchte nach einer Nutzung/Monetarisierung außerhalb der Spitzen → AWS (seit 2002).

c) Amazon (AWS), Microsoft (Azure), Google Cloud (GCP), STACKIT (Schwarz-Gruppe, Neckarsulm).

d) Infrastructure as a Service · Platform as a Service · Software as a Service (Pionier: Salesforce, CRM). Details folgen im 2. Semester.

Block 3: Lösungen Mooresches Gesetz

Klausur-Aufgabe 3 (2 P)

P3.1

- ☐ Die Anzahl der Transistoren auf einem Chip verdreifacht sich alle 1–2 Jahre.
  falsch: verdoppelt, nicht verdreifacht.
- ☒ Die Leistung von Mikroprozessoren verdoppelt sich alle ca. 18 Monate.
  richtig: eine der drei Interpretationen.
- ☐ Die in einem Computer verbaute Speicherkapazität verdoppelt sich regelmäßig alle 18 Monate.
  falsch: Moore betrifft Transistoren/Leistung/Kosten, nicht die verbaute Speicherkapazität.
- ☐ Die Größe eines Transistors halbiert sich alle 1–2 Jahre.
  falsch: Nicht die Größe, sondern die Anzahl pro Chip (Verdopplung) ist Gegenstand.

Laut Lösungsfolie ist nur die zweite Aussage richtig. Die Fallen stecken in einzelnen Wörtern.

P3.2

2020 → 2024 = 2 Verdopplungen: 54 Mrd. · 2^2 = **216 Mrd.** – Blackwell liegt mit 208 Mrd. sehr nahe daran. In 10 Jahren: 5 Verdopplungen → Faktor 2^5 = **32**.

P3.3

Grenzen: Bei immer kleineren Strukturen steigen die Kosten pro Transistor wieder; Energiebedarf und Kühlungsaufwand wachsen mit der Rechenleistung (Sättigung durch Temperatur und Kosten). Ansätze: neue Materialien, Prozesse und Lithographie-Verfahren (ASML, TSMC), Chipdesign in der 3. Dimension, Quantencomputer, Skalierung über mehrere GPUs/Cluster/Rechenzentren.

Block 4: Lösungen Zahlendarstellung und Rechnen

Klausur-Aufgabe 4 (3 P) – „Übungssache!“

P4.1

Dezimal	Dual	Oktal	Hexadezimal
200	1100 1000	310	C8
3045	1011 1110 0101	5745	BE5
511	0001 1111 1111	777	1FF
1015	0011 1111 0111	1767	3F7
1000	0011 1110 1000	1750	3E8

Beispielweg 200: 200 : 2 = 100 Rest 0, 100 : 2 = 50 Rest 0, 50 : 2 = 25 Rest 0, 25 : 2 = 12 Rest 1, 12 : 2 = 6 Rest 0, 6 : 2 = 3 Rest 0, 3 : 2 = 1 Rest 1, 1 : 2 = 0 Rest 1 → Reste von unten: 1100 1000. Oktal/Hex über 3er/4er-Gruppen von rechts; $777_8 = 7{\cdot}64 + 7{\cdot}8 + 7 = 511$; $3F7_{16} = 3{\cdot}256 + 15{\cdot}16 + 7 = 1015$.

P4.2

$2F0_{16} = 752$, $1357_8 = 751$, $753_{10} = 753$, $10\ 1111\ 0010_2 = 754$ → **$1357_8 < 2F0_{16} < 753_{10} < 10\ 1111\ 0010_2$**.

Folienaufgabe: $E7E5_{16} = 59365$, $1110\ 0111\ 1110\ 0110_2 = E7E6_{16} = 59366$ → der Binärwert ist am größten.

P4.3

Aufgabe	dezimal	binäres Ergebnis	Überlauf (8 Bit)?
0100 0001 + 0101 1010	65 + 90 = 155	1001 1011	nein
0111 1111 + 0111 1111	127 + 127 = 254	1111 1110	nein
1011 0110 + 0110 1101	182 + 109 = 291	0001 0010 0011	ja – 9. Bit, Ergebnis > 255

c) In 8 Bit bleibt nur `0010 0011` = 35 stehen (291 − 256); der Wertebereich 0 … 255 ist überschritten.

P4.4

a) +42 = 0010 1010.

Darstellung	−42	Wertebereich
Vorzeichen und Betrag	1010 1010	−127 … +127, zwei Nullen
Einerkomplement	1101 0101	−127 … +127, zwei Nullen, End-around-Carry
Zweierkomplement	1101 0110	−128 … +127, eine Null

b) 42 = `0010 1010`, -17 = `1110 1111`, 0 = `0000 0000`, -1 = `1111 1111`, -128 = `1000 0000`, 127 = `0111 1111`.

c) Nur eine Null, größerer Wertebereich (−128), und Addition wie Subtraktion laufen über dieselbe Addierschaltung (letzter Übertrag wird ignoriert). Das höchste Bit hat die Wertigkeit −128: −128 + 64 + 16 + 4 + 2 = −42.

P4.5

Rechnung	Operand 1	Operand 2	8-Bit-Ergebnis	Wert	soll
42 + (-17)	00101010	11101111	0001 1001	25 ✓	25
-42 + (-17)	11010110	11101111	1100 0101	-59 ✓	-59
-1 + (-17)	11111111	11101111	1110 1110	-18 ✓	-18
42 + (-42)	00101010	11010110	0000 0000	0 ✓	0
100 + 50	01100100	00110010	1001 0110	-106 → **Überlauf**	150
-100 + (-50)	10011100	11001110	0110 1010	106 → **Überlauf**	-150

Überlaufregel: Haben beide Operanden dasselbe Vorzeichen und das Ergebnis ein anderes, liegt ein Überlauf vor (150 > 127 bzw. −150 < −128). Ein Übertrag aus Bit 7 allein ist *kein* Fehler (z. B. 42 − 17), er wird ignoriert.

P4.6

a) vorzeichenlos 128 + 32 + 16 + 4 + 1 = **181**; Zweierkomplement −128 + 32 + 16 + 4 + 1 = **−75**.

b) −32 + 8 + 4 + 1 = **−19**.

c) 9 = 01001 → −9 = 10111; 01110 + 10111 = (1)00101 = **5** (Übertrag ignoriert).

d) n Bit: -2^{n-1} … $2^{n-1} - 1$ → 4 Bit: −8 … 7; 16 Bit: −32768 … 32767; 32 Bit: −2.147.483.648 … 2.147.483.647.

P4.7

a) 6 = 110; Nachkomma: 0,625 · 2 = 1,25 → 1; 0,25 · 2 = 0,5 → 0; 0,5 · 2 = 1,0 → 1 → $6{,}625_{10}$ = **$110{,}101_2$** (= 4 + 2 + ½ + ⅛).

b) 4 + 1 + ¼ + ⅛ = **5,375**.

c) 0,1·2 = 0,2 → 0; 0,4 → 0; 0,8 → 0; 1,6 → 1; 1,2 → 1; 0,4 → 0 … ab hier wiederholt sich die Folge: $0{,}1_{10}$ = $0{,}000110011001\ldots_2$ (periodisch 0011). Mit endlich vielen Bits muss abgeschnitten werden → Rundungsfehler.

d) Mantisse m (meist nicht ganzzahliger Anteil), Basis b, Exponent e: 2022 = 2,022 · 10^3 (m = 2,022; b = 10; e = 3). Vorteil: nicht ganzzahlige Werte und ein viel größerer Zahlenbereich.

e) Vorzeichen 1 (negativ). $6{,}625 = 110{,}101_2 = 1{,}10101 \cdot 2^2$ → Exponent 2 + 127 = 129 = `10000001`. Mantisse ohne „Hidden Bit“: `10101000000000000000000`.
Ergebnis: `1 10000001 10101000000000000000000` = 0xC0D40000. (Folienbeispiel 18,4: 0 10000011 00100110011001100110011 – Ungenauigkeit durch Abschneiden der Mantisse.)

f) 23 = 0001 0111; << 2 → 0101 1100 = **92** (= ·4); 92 >> 1 → 0010 1110 = **46** (= :2).

Block 5: Lösungen Wahrheitstabellen, Boolesche Ausdrücke, Logikgatter

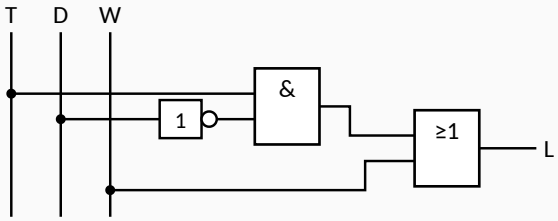
Klausur-Aufgabe 5 (4 P) – „Auch Übungssache!“

P5.1

T	D	W	L
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

b) $L = (T \wedge \neg D) \vee W$

DNF: $(\neg T \wedge \neg D \wedge W) \vee (\neg T \wedge D \wedge W) \vee (T \wedge \neg D \wedge \neg W) \vee (T \wedge \neg D \wedge W) \vee (T \wedge D \wedge W)$



P5.2

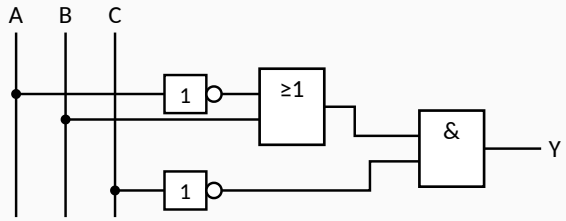
1: $A \wedge \neg B$ · 2: $B \wedge \neg A$ · 3: $(\neg A \wedge B) \vee (A \wedge \neg B)$ = A XOR B („ungleich“) · 4: $\neg(A \wedge B)$ = A NAND B = $\neg A \vee \neg B$ (De Morgan).

Standardvorgehen: Für jede Zeile mit Out = 1 einen UND-Term bilden (Variable mit 0 negiert), alle Terme mit ODER verknüpfen.

P5.3

DNF: $Y = (\neg A \wedge \neg B \wedge \neg C) \vee (\neg A \wedge B \wedge \neg C) \vee (A \wedge B \wedge \neg C)$

Vereinfachung: alle Terme enthalten $\neg C \rightarrow Y = \neg C \wedge [(\neg A \wedge \neg B) \vee (\neg A \wedge B) \vee (A \wedge B)] = \neg C \wedge [\neg A \vee (A \wedge B)] =$ **$\neg C \wedge (\neg A \vee B)$** (per Wertetabelle geprüft).



Eine Schaltung direkt aus der DNF (drei UND-Gatter mit je drei Eingängen, NOT-Gatter, ein ODER-Gatter) ist genauso richtig.

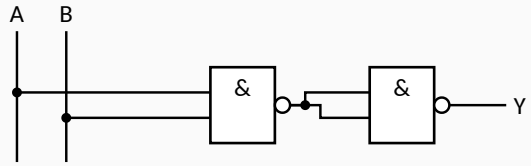
P5.4

Y = ¬(A ∧ B) ∧ (B ∨ C)

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

P5.5

a) A NAND B, danach dasselbe Signal auf beide Eingänge eines zweiten NAND (wirkt als Inverter): ¬(¬(A∧B)) = A ∧ B.



b) Beide Eingänge verbinden: ¬(A ∧ A) = ¬A.

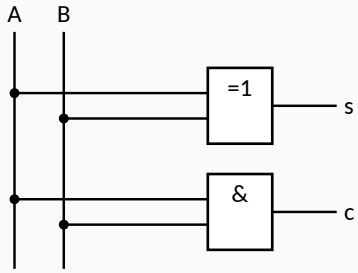
c)

A	B	¬(A∧B)	¬A∨¬B	¬(A∨B)	¬A∧¬B
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

Die Spalten stimmen paarweise überein. Wegen dieser Vollständigkeit genügt in der Chipfertigung (und in FPGAs) ein Gattertyp.

P5.6

A	B	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



s = A XOR B, c = A ∧ B

b) Volladdierer: addiert zwei Bits plus eingehenden Übertrag; aufgebaut aus zwei Halbaddierern und einem ODER-Gatter (für die beiden Teilüberträge). Addierwerk: mehrere Volladdierer hintereinander, der Übertrag wird jeweils an die nächsthöhere Stelle weitergereicht.

c) Bit 0: 0+1 = 1, c=0 · Bit 1: 1+1 = 0, c=1 · Bit 2: 1+1+1 = 1, c=1 · Bit 3: 0+0+1 = 1, c=0 → **1101** (6 + 7 = 13).

Block 6: Lösungen Programmierung

Klausur-Aufgabe 6 (2 P)

P6.1

Algorithmus: Handlungsvorschrift zur Lösung eines Problems in endlicher Zeit (Kuchenrezept, Sortieren). **Programm:** Umsetzung eines oder mehrerer Algorithmen in einer Programmiersprache. **Prozess:** ein laufendes Programm mit seinen Daten, Variablenwerten und Verwaltungsinformationen des Betriebssystems. Die Endlos-Ausgabe terminiert nie → streng genommen **kein Algorithmus**, aber ein gültiges Programm (wie ein Serverprozess).

P6.2

a) fakrek(5) = 5 · fakrek(4) = 5 · 4 · fakrek(3) = 5 · 4 · 3 · fakrek(2) = 5 · 4 · 3 · 2 · fakrek(1) = 5 · 4 · 3 · 2 · 1 = **120**; 5 Aufrufe, der letzte (number == 1) bricht ab.

b) Ohne Abbruchbedingung ruft sich die Funktion endlos selbst auf (bis der Aufrufstapel überläuft).

```
sum = 1
für i = 1 bis number: sum = sum * i
return sum
```

P6.3

Beschreibung	Werkzeug
übersetzt das Quellprogramm vorab in Maschinenbefehle für eine bestimmte Hardware	Compiler
übersetzt Befehle erst zur Laufzeit und führt sie aus (ggf. über Bytecode)	Interpreter
verbindet getrennt übersetzte Module/Bibliotheken zu einem ausführbaren Programm (statisch oder dynamisch, DLL)	Linker
lädt den Maschinencode an eine Speicheradresse und passt die Adressen an (beim Start: Bootstrap)	Lader
führt ein Programm schrittweise mit Haltepunkten aus	Debugger
vereint Editor, Compiler und Debugger (z. B. Eclipse)	IDE

Phasen eines Compilers: Scanner (lexikalische Analyse) → Parser (Syntax) → semantische Analyse/Optimierung → Codeerzeugung.

P6.4

- a) Syntaxfehler – Klammer fehlt, Compiler meldet Fehler
- b) Semantikfehler – Name passt nicht zum Inhalt, keine Meldung
- c) Typ-Falle (Semantik): int/int = Ganzzahldivision → r = 3.0; richtig (double) z1 / z2 = 3.5
- d) Syntaxfehler – Semikolon fehlt

Block 7: Lösungen Datenstrukturen

Klausur-Aufgabe 7 (6–8 P)

P7.1

Operation	Rückgabe	Stapel (unten → oben)
push(8)	–	[8]
push(2)	–	[8, 2]
top()	2	[8, 2]
push(6)	–	[8, 2, 6]
pop()	6	[8, 2]
pop()	2	[8]
pop()	8	[]
pop()	**Fehler**: Stapel leer	[]

Das letzte pop() trifft einen leeren Stapel → Fehlerfall (Ausnahme/Fehlermeldung, vorher mit „leer?“ prüfen).
b) D, H, B, W nacheinander pushen, dann viermal pop() → W, B, H, D = „WBHD“.

P7.2

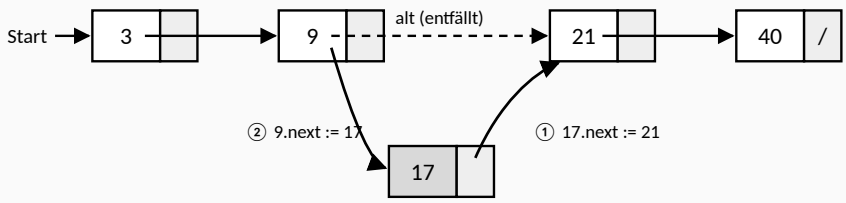
Operation	Rückgabe	Queue (Anfang → Ende)
put(A)	–	[A]
put(B)	–	[A, B]
put(C)	–	[A, B, C]
get()	A	[B, C]
put(D)	–	[B, C, D]
get()	B	[C, D]
get()	C	[D]
put(E)	–	[D, E]

Gedruckt: A, B, C; übrig: D, E. get() auf leere Queue = Fehlerfall.

P7.3

get() liefert 1, 2, 3, 4 (FIFO) → Stapel unten 1 … oben 4 → pop() liefert **4, 3, 2, 1** (LIFO).

P7.4



Erst den neuen Knoten auf den Nachfolger zeigen lassen (①), dann den Vorgänger umhängen (②).

a) Ab Start durchlaufen, bis der Vorgänger gefunden ist (9, da 21 > 17). ① 17.next := 21, ② 9.next := 17 → 3 → 9 → 17 → 21 → 40.

b) Vorgänger von 21 suchen (17), 17.next := 40 (= 21.next). 21 ist danach nicht mehr erreichbar.

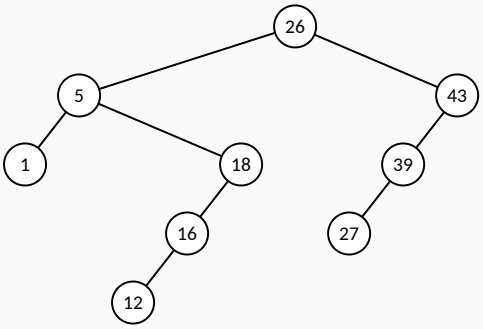
c) Jedes Element kennt Vorgänger und Nachfolger – kennt man das zu löschende Element, muss man die Liste nicht durchlaufen, um den Vorgänger zu finden.

d) Im Array (feste Länge) müssen alle größeren Elemente nach hinten umkopiert werden (ggf. in ein neues, größeres Array); in der Liste werden nur zwei Zeiger geändert. Dafür erlaubt das Array direkten Zugriff per Index.

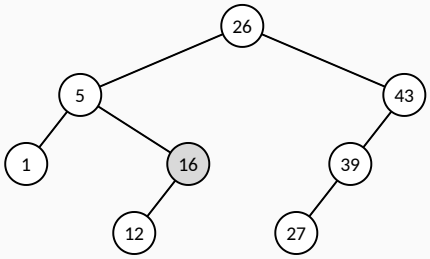
P7.5

a) Jeder Knoten hat höchstens zwei Nachfolger (links/rechts); geordnet: linker Nachfolger kleiner, rechter größer als der Knoten. Vereint schnelles Finden (wie Array) mit leichtem Einfügen/Löschen (wie Liste).

b)



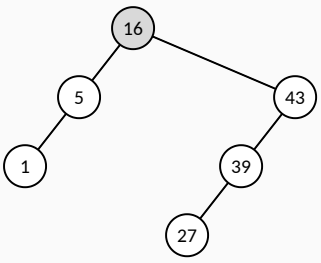
c) 18 hat nur einen Nachfolger (16) → 16 rückt nach:



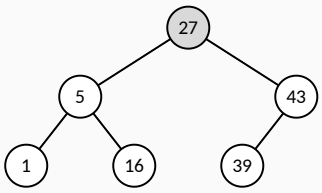
Nach c): **weder voll noch vollständig** (Halbblätter 16, 43, 39 mit nur einem Nachfolger; Blätter auf verschiedenen Ebenen).

d) 26 hat zwei Nachfolger → ersetzen durch den größten Knoten links (16) *oder* den kleinsten rechts (27) – beide Varianten sind nach der Folienregel korrekt.

Variante „größter links“, nach Löschen von 26 und 12:



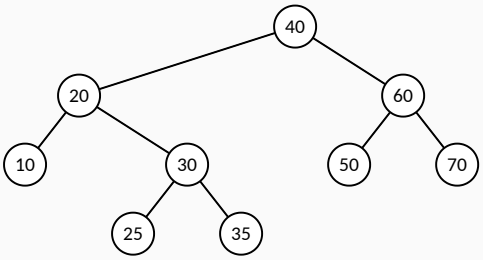
Variante „kleinster rechts“:



In beiden Varianten: **weder voll noch vollständig** (es gibt Knoten mit genau einem Nachfolger, z. B. 43 bzw. 5).

Degenerierter Binärbaum: jeder Knoten hat höchstens einen Nachfolger → praktisch eine Liste, Suchaufwand O(n) (entsteht z. B. beim Einfügen sortierter Werte).

P7.6



Wurzel 40 · Blätter 10, 25, 35, 50, 70 · innere Knoten 20, 30, 40, 60 · Ebene von 25: 3 (Kanten bis zur Wurzel) · Höhe 3 · Pfadlänge = Summe aller Ebenen = 0 + 2·1 + 4·2 + 2·3 = **16**.

Voll: ja – jeder innere Knoten hat genau zwei Nachfolger. **Vollständig:** nein – die Blätter 10, 50, 70 liegen nicht auf der letzten Ebene. Suche nach 35: Vergleiche mit 40 → 20 → 30 → 35 = 4 Vergleiche.

Block 8: Lösungen Suchen und Sortieren

Klausur-Aufgabe 8 (6 P)

P8.1

Vergleich	Tausch?	Array danach
7 ↔ 3	ja	[3, 7, 9, 1, 5]
7 ↔ 9	nein	[3, 7, 9, 1, 5]
9 ↔ 1	ja	[3, 7, 1, 9, 5]
9 ↔ 5	ja	[3, 7, 1, 5, 9]

Durchlauf	Array	Vertauschungen
1	[3, 7, 1, 5, 9]	3
2	[3, 1, 5, 7, 9]	2
3	[1, 3, 5, 7, 9]	1
4	[1, 3, 5, 7, 9]	0

Insgesamt 6 Vertauschungen. Nach Durchlauf 1 steht 9 (das Maximum) am Ende; jeder Durchlauf bringt das nächstgrößte Element an seinen Platz.

P8.2

```
Teilen 0: [6 2 9 4 1 8 3]
Teilen 1: [6 2 9]  [4 1 8 3]
Teilen 2: [6]  [2 9]  [4 1]  [8 3]
Teilen 3: [6]  [2]  [9]  [4]  [1]  [8]  [3]
Mischen 1: [6]  [2 9]  [1 4]  [3 8]
Mischen 2: [2 6 9]  [1 3 4 8]
Mischen 3: [1 2 3 4 6 8 9]
```

Teile und Herrsche: Aufgabe in zwei gleichartige Teilaufgaben halber Größe zerlegen, diese lösen, Ergebnisse zusammenführen. Rekursion ist die Umsetzung: mergesort ruft sich für beide Hälften selbst auf (Abbruch bei Länge 1). Aufwand O(n log n).

P8.3

- 61: [0…8] Mitte 4 = 36 → [5…8] Mitte 6 = 79 → [5…5] Mitte 5 = 61 → gefunden
- 19: [0…8] Mitte 4 = 36 → [0…3] Mitte 1 = 19 → gefunden
- 80: [0…8] Mitte 4 = 36 → [5…8] Mitte 6 = 79 → [7…8] Mitte 7 = 81 → Bereich leer, **nicht gefunden**

Hinweis: Die Folie zeigt für 19 drei Schritte (36 → 23 → 19); dort wird in der linken Hälfte die obere Mitte gewählt. In der Klausur zählt ein konsequent angewandtes Vorgehen – die Mitte-Regel am besten kurz dazuschreiben.

P8.4

n	linear (max.)	binär (max.)
16	16	5
1.000	1.000	10
1.000.000	1.000.000	20

Binär: etwa $\lfloor \log_2 n \rfloor + 1$ Vergleiche ($2^{20} \approx 1$ Mio.). Nur linear möglich bei **unsortierten** Daten bzw. wenn kein direkter Zugriff auf die Mitte besteht (verkettete Liste).

P8.5

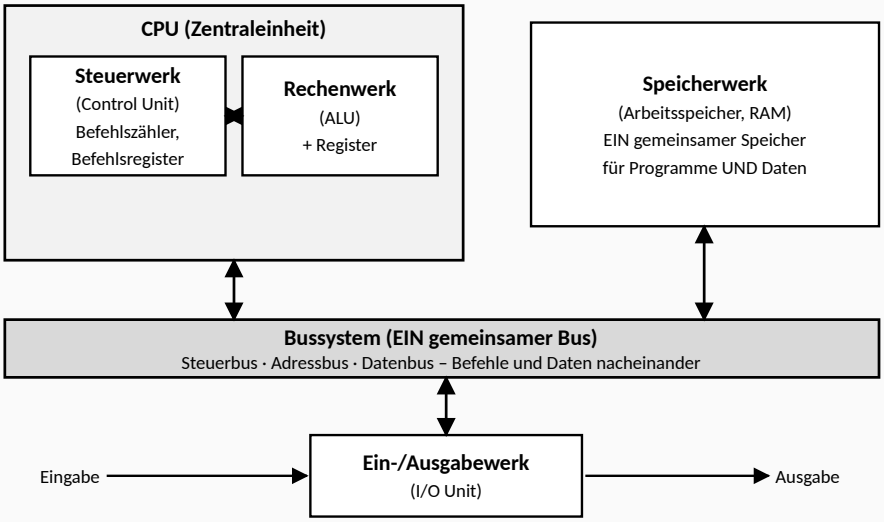
Verfahren	Aufwand	Merkmal
Bubble Sort	$O(n^2)$	benachbarte Elemente vergleichen/tauschen; naiv
Merge Sort	O(n log n)	Teile und Herrsche, rekursiv, Zusammenführen
lineare Suche	O(n)	auch unsortiert
binäre Suche	O(log n)	nur sortiert, Halbierung

Auf der Folie zusätzlich genannt: Quicksort (meistgenutzt, ohne Details).

Block 9: Lösungen Rechnerarchitektur

Klausur-Aufgabe 9 (8 P)

P9.1



- **Rechenwerk (ALU):** Rechenoperationen und boolesche Verknüpfungen.
- **Steuerwerk (Control Unit):** interpretiert die Befehle, verschaltet Datenquelle, Datensenke und ALU, regelt die Befehlsfolge.
- **Bussystem:** Kommunikation zwischen den Komponenten (Steuer-, Adress-, Datenbus).
- **Speicherwerk (RAM):** speichert Programme *und* Daten.
- **Ein-/Ausgabewerk:** Ein-/Ausgabe zum Anwender (Tastatur, Bildschirm) oder zu anderen Systemen.

P9.2

1. **FETCH:** Befehl von der Adresse im Befehlszähler aus dem Speicher ins Befehlsregister laden.
2. **DECODE:** Steuerwerk erkennt „Lade in den Akkumulator“ und erzeugt die Schaltinstruktionen.
3. **FETCH OPERANDS:** Wert von Adresse NUM1 aus dem Speicher holen.
4. **EXECUTE:** Wert in den Akkumulator schreiben.
5. **UPDATE INSTRUCTION POINTER:** Befehlszähler zeigt auf den nächsten Befehl (CLC). Dann beginnt der Zyklus von vorn.

Jede Phase kann mehrere Takte dauern; Befehl und Operand laufen nacheinander über denselben Bus.

P9.3

Vorteile Von-Neumann: einfache Implementierung, geringere Kosten; Universalrechner – Programme liegen wie Daten im Speicher und können zur Laufzeit geladen und geändert werden.

Nachteile: Von-Neumann-Flaschenhals – Befehle und Daten müssen nacheinander über den gemeinsamen Bus, mindestens 2 Taktzyklen pro Anweisung. Sicherheit – weil Code und Daten im selben Speicher liegen, kann als Daten getarnter Code eingeschleust werden (z. B. über Buffer-Überläufe).

Harvard: getrennte Speicher und Busse, Befehl und Daten in einem Schritt, schneller – aber in der Regel teurer. Einsatz daher v. a. in Mikrocontrollern, Signalverarbeitung, KI-Spezialrechnern, zunehmend in PCs/Servern.

P9.4

a) Befehlszähler: verweist auf (enthält die Adresse von) den nächsten auszuführenden Befehl. Befehlsregister: enthält den momentan auszuführenden Befehl selbst.

b) Befehle aus dem Hauptspeicher abrufen und Befehlstyp bestimmen; Operanden in Register laden; Rechenwerk zur Operation veranlassen; bestimmen, wo es (ggf. abhängig vom Ergebnis) weitergeht.

c) Kleiner Hochgeschwindigkeitsspeicher in der CPU für jeweils einen Wert; Daten-Schnittstelle zum Rechenwerk.

P9.5

- [x] Beim Von-Neumann-Rechner liegen Programme und Daten im selben Speicher.
  richtig: Kernmerkmal.
- [x] Die Harvard-Architektur verwendet getrennte Busse für Befehle und Daten.
  richtig: Daher Befehl und Daten in einem Schritt.
- [ ] Der Adressbus überträgt bei einem Lesezugriff die gelesenen Daten zur CPU.
  falsch: Daten laufen über den Datenbus; der Adressbus trägt die Adresse.
- [x] Peripherie wird in der Regel seriell angebunden, z. B. über USB.
  richtig: USB = Universal Serial Bus; Adress-/Datenbus sind parallel.
- [ ] Die Harvard-Architektur ist in der Regel billiger als die Von-Neumann-Architektur.
  falsch: Umgekehrt: Harvard ist in der Regel teurer.
- [x] Der Von-Neumann-Zyklus besteht aus FETCH, DECODE, FETCH OPERANDS, EXECUTE und UPDATE INSTRUCTION POINTER.
  richtig: Fünf Phasen laut Folie.

P9.6

Steuerbus: Schreibbefehl · Adressbus: Adresse der Speicherzelle · Datenbus: Daten von der CPU zur Speicherzelle. Parallel: viele Leitungen übertragen gleichzeitig mehrere Bits (z. B. Apple M1: 64-Bit-Adressbus, 128-Bit-Datenbus); seriell: Bit für Bit nacheinander.

Block 10: Lösungen Speicher

Klausur-Aufgabe 10 (4 P)

P10.1

n	adressierbarer Speicher
16	2^16 = 65.536 Byte = 64 KB
24	2^24 = 16.777.216 Byte = 16 MB
32	2^32 = 4.294.967.296 Byte = 4 GB
39	512 GB = 2^39 Byte
33	8 GB = 2^33 Byte
48	2^48 = 281.474.976.710.656 Byte = 256 TB

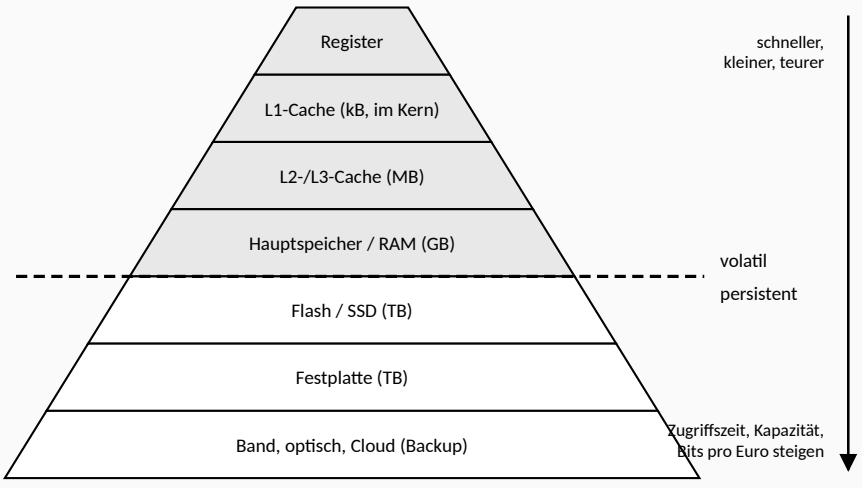
Merke: n Leitungen → 2^n Zellen (nicht n · 8). 2^{10} = 1 K, 2^{20} = 1 M, 2^{30} = 1 G, 2^{40} = 1 T.

P10.2

Level	Platten	nutzbar	Ausfälle
RAID 1	2 × 4 TB	4 TB	1
RAID 5	3 × 4 TB	8 TB	1
RAID 0	5 × 4 TB	20 TB	0
RAID 5	5 × 4 TB	16 TB	1
RAID 6	5 × 4 TB	12 TB	2
RAID 6	8 × 1 TB	6 TB	2

RAID 1 (Mirroring): halbe Kapazität bei zwei Platten. RAID 5: mind. 3 Platten, n − 1 nutzbar. RAID 6: mind. 4 Platten, n − 2 nutzbar. RAID ersetzt **kein Backup** (versehentliches Löschen, Schadsoftware, Brand betreffen alle Platten).

P10.3



Nach unten wachsen **Zugriffszeit**, **Speicherkapazität** und **Bits pro Geldeinheit** (langsamer, größer, billiger). Volatil bis einschließlich RAM, persistent ab Flash.

P10.4

Die CPU wird immer schneller als der Hauptspeicher; den ganzen Speicher auf die CPU zu bringen wäre zu teuer. Kompromiss: kleiner, sehr schneller Cache in bzw. vor der CPU (L1–L3). Idee **Lokalität**: Programme greifen in kurzer Zeit nur auf einen kleinen Teil des Speichers zu, deshalb werden beim Zugriff auf ein Wort auch dessen Nachbarn in den Cache geladen. Beispiele: Schleifen (derselbe Code mehrfach), Arrays/Tabellen (benachbarte Datensätze). Cache-Miss: Der Bereich liegt wider Erwarten nicht im Cache und muss über den Bus aus dem Speicher nachgeladen werden.

P10.5

- [x] Der Inhalt des Arbeitsspeichers (RAM) geht ohne Strom verloren.
  richtig: RAM ist volatil.
- [x] Flash-Speicher (früher ROM/EPROM) behält seinen Inhalt auch stromlos.
  richtig: persistent; z. B. BIOS/Firmware.
- [ ] Register sind langsamer als der L1-Cache.
  falsch: Register sind der schnellste Speicher.
- [x] Die kleinste über den Adressbus adressierbare Einheit ist üblicherweise das Byte.
  richtig: 8-Bit-Zelle.
- [ ] RAID 0 spiegelt die Daten auf eine zweite Platte.
  falsch: Das ist RAID 1; RAID 0 = Striping ohne Redundanz.
- [x] Beim Harvard-Rechner werden Daten- und Programmspeicher getrennt adressiert.
  richtig: Folie Speicherarten.

Block 11: Lösungen Datenbanken

Klausur-Aufgabe 11 (4–6 P)

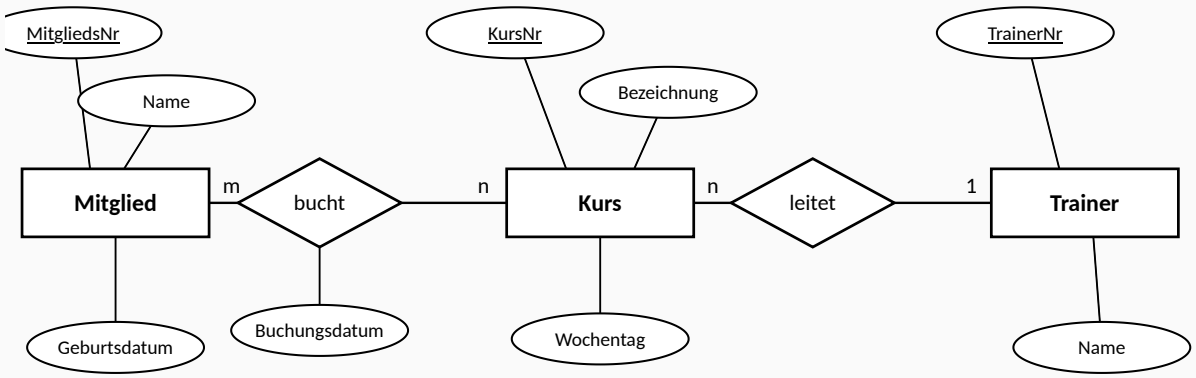
P11.1

a) Redundanz und Inkonsistenz (z. B. Adressänderung nur in der Buchhaltung) · Abhängigkeit zwischen Programm und Daten · fehlender Datenaustausch / mangelnde Flexibilität · mangelnde Datensicherheit (Zugriffsrechte).

b) Datenbank: Sammlung logisch zusammenhängender Daten mit Bedeutung, für einen Zweck. DBMS: Software zum Anlegen und Pflegen der Datenbank. Datenbanksystem: DB + DBMS (+ Anwendung). Data Dictionary: Datenbankkatalog mit Metadaten (welche Tabellen, Attribute, Datentypen, Einschränkungen).

c) Daten und Anwendung einfach und stabil · Echtzeitanforderungen, die ein DBMS nicht erfüllt · kein Mehrbenutzerzugriff nötig.

P11.2



b) MITGLIED(MitgliedsNr, Name, Geburtsdatum) · TRAINER(TrainerNr, Name) · KURS(KursNr, Bezeichnung, Wochentag, TrainerNr → TRAINER) · BUCHUNG(MitgliedsNr → MITGLIED, KursNr → KURS, Buchungsdatum).

Regeln: Entitätstyp → Tabelle; 1:n → Fremdschlüssel auf der n-Seite (KURS); m:n → eigene Verknüpfungstabelle mit beiden Fremdschlüsseln als zusammengesetztem Primärschlüssel; Beziehungsattribute (Buchungsdatum) wandern in diese Tabelle.

P11.3

Beziehung	Kardinalität	Umsetzung
Person – Personalausweis (gültig)	1:1	Fremdschlüssel auf einer Seite
Mutter – Kind	1:n	Fremdschlüssel auf der n-Seite
Student – Vorlesung (besucht)	m:n	Hilfs-/Verknüpfungstabelle
Abteilung – Leiter/in	1:1	Fremdschlüssel auf einer Seite
Rechnung – Rechnungsposten	1:n	Fremdschlüssel auf der n-Seite
Kunde – Artikel (bestellt)	m:n	Hilfs-/Verknüpfungstabelle

P11.4

a) `SELECT VNAME, NNAME FROM ANGESTELLTER WHERE GEHALT > 30000 AND ABT = 5;`

VNAME	NNAME
Franklin	Wong
Ramesh	Narayan

b) `SELECT ABT, COUNT(*), AVG(GEHALT) FROM ANGESTELLTER GROUP BY ABT;`

ABT	COUNT(*)	AVG(GEHALT)
1	1	55000
4	3	31000
5	4	33250

c) `SELECT PNAME, PSTANDORT FROM PROJEKT, ABTEILUNG WHERE ABTNR = ABTNUMMER AND ANAME = 'Administration';`

PNAME	PSTANDORT
Computerization	Stafford
Newbenefits	Stafford

d) `SELECT SUM(STUNDEN), MAX(STUNDEN) FROM ARBEITET_AN WHERE PNR = 10;`

SUM(STUNDEN)	MAX(STUNDEN)
55	35

e) `SELECT PNUMMER, ABTNR, NNAME FROM PROJEKT, ABTEILUNG, ANGESTELLTER WHERE ABTNR = ABTNUMMER AND MGRSSN = SSN AND PSTANDORT = 'Stafford';`

PNUMMER	ABTNR	NNAME
10	4	Wallace
30	4	Wallace

a) Angestellte der Abt. 5 mit mehr als 30000 Gehalt · b) Anzahl und Durchschnittsgehalt je Abteilung · c) Projekte der Abteilung Administration · d) Stunden-Summe und -Maximum für Projekt 10 · e) Folienbeispiel: Projekte in Stafford mit Abteilung und Nachname des Abteilungsleiters (zwei Joins).

P11.5

a)

```
SELECT VNAME, NNAME
FROM   ANGESTELLTER
WHERE  GESCHLECHT = 'F';
```

VNAME	NNAME
Alicia	Zelaya
Jennifer	Wallace
Joyce	English

b)

```
SELECT MAX(GEHALT), MIN(GEHALT)
FROM   ANGESTELLTER
WHERE  ABT = 4;
```

MAX(GEHALT)	MIN(GEHALT)
43000	25000

c)

```
SELECT NNAME
FROM   ANGESTELLTER, ARBEITET_AN, PROJEKT
WHERE  SSN = ESSN
  AND  PNR = PNUMMER
  AND  PNAME = 'ProductX';
```

NNAME
Smith
English

d)

```
SELECT PSTANDORT, COUNT(*) AS Anzahl
FROM   PROJEKT
GROUP BY PSTANDORT;
```

PSTANDORT	Anzahl
Bellaire	1
Houston	2
Stafford	2
Sugarland	1

e)

```
SELECT ANAME, NNAME
FROM   ABTEILUNG, ANGESTELLTER
WHERE  MGRSSN = SSN;
```

ANAME	NNAME
Headquarters	Borg
Administration	Wallace
Research	Wong

f)

```
SELECT ANAME, SUM(GEHALT) AS Summe
FROM   ANGESTELLTER, ABTEILUNG
WHERE  ABT = ABTNUMMER
GROUP BY ANAME;
```

ANAME	Summe
Administration	93000
Headquarters	55000
Research	133000

Die Reihenfolge der Ergebniszeilen ist ohne ORDER BY nicht festgelegt. Gleichwertig: JOIN … ON statt Komma + WHERE.

P11.6

a)

```
CREATE TABLE KURS
( KursNr      int PRIMARY KEY,
  Bezeichnung char(40) not null,
  Wochentag   char(10) );
```

b)

```
INSERT INTO PROJEKT (PNAME, PNUMMER, PSTANDORT, ABTNR)
VALUES ('ProductW', 40, 'Houston', 5);
```

c)

```
UPDATE ANGESTELLTER
SET    GEHALT = GEHALT * 1.1
WHERE  ABT = 4;
```

d)

```
DELETE FROM PROJEKT
WHERE  PNUMMER = 30;
```

Ergebnis c) (geprüft): Abt. 4 hat danach 27500, 27500 und 47300. d) In ARBEITET_AN verweisen dann noch Zeilen mit PNR = 30 ins Leere – genau solche Beziehungen überwacht ein DBMS über Integritätsbedingungen.

e) Ohne WHERE wirken UPDATE und DELETE auf **alle** Zeilen der Tabelle.

P11.7

a) Relation = Tabelle (nicht: Beziehung!) · Tupel = Zeile/Datensatz · Attribut = Spalte · Domäne = Wertebereich/Datentyp eines Attributs (z. B. Alter 15–80) · atomar = Attributwert nicht weiter zerlegbar (keine Liste von Telefonnummern in einer Zelle).

b) konzeptuell: ER-Modell (DBMS-unabhängig) · logisch: relationale Tabellen · physisch: Speicherung in Dateien, Indizes.

	OLTP	OLAP
Zweck	operatives Tagesgeschäft, Transaktionen	Analyse, BI, Data Warehouse
Zugriff	Lesen und Schreiben, aktuelle Daten	(fast) nur Lesen, inkl. Historie
Speicherung	zeilenweise (Row Store)	spaltenweise (Column Store), gut komprimierbar, In-Memory
Beispiele	Oracle, MySQL	Amazon Redshift, SAP HANA

Daten gelangen per ETL (Extract, Transform, Load) vom OLTP- ins OLAP-System.